



C#-Other OOP Concepts

- Sealed Classes
- Sealed methods
- Static Members of a Class
- Static Classes
- Static Methods
- Partial classes
- Partial methods



- Sealed classes are used to restrict the users from inheriting the class.
- A class can be sealed by using the ***sealed*** keyword.
- The keyword tells the compiler that the class is sealed, and therefore, cannot be extended. No class can be derived from a sealed class.

The following is the **syntax** of a sealed class :

```
sealed class class_name  
{  
    // data members  
    // methods  
}
```

- *A method can also be sealed*, and in that case, the method cannot be overridden. However, a method can be sealed in the classes in which they have been inherited. If you want to declare a method as sealed, then it has to be declared as **virtual** in its base class.

```
// Sealed class
sealed class SealedClass
{
    // Calling Function
    public int Add(int a, int b)
    {
        return a + b;
    }
}

class Program
{
    // Main Method
    static void Main(string[] args)
    {
        // Creating an object of Sealed Class
        SealedClass slc = new SealedClass();

        // Performing Addition operation
        int total = slc.Add(6, 4);
        Console.WriteLine("Total = " + total.ToString());
    }
}
```

if it tries to inherit a class from a sealed class then an error will be produced stating that " It cannot be derived from a **Sealed class** .

- **Important point regarding Sealed Class:**
- Sealed class is used to stop a class to be inherited. You cannot derive or extend any class from it.
- Sealed method is implemented so that no other class can overthrow it and implement its own method.
- The main purpose of the sealed class is to withdraw the inheritance attribute from the user so that they can't attain a class from a sealed class. Sealed classes are used best when you have a class with static members.
e.g the "Pens" and "Brushes" classes of the System.Drawing namespace. The Pens class represents the pens for standard colors. This class has only static members.

- In C#, static means something which cannot be instantiated. You cannot create an object of a static class and cannot access static members using an object.
- C# classes, variables, methods, properties, operators, events, and constructors can be defined as static using the static modifier keyword.

```
public static class Calculator
{
    public static string Type = "Arithmetic";

    public static int Sum(int num1, int num2)
    {
        return num1 + num2;
    }
}

class Program
{
    static void Main(string[] args)
    {
        var result = Calculator.Sum(10, 25); // calling static method
        Console.WriteLine(result);

        var calcType = Calculator.Type; // accessing static variable
        Calculator.Type = "Scientific"; // assign value to static variable
    }
}
```

Calculator class is a static. All the members of it are also static. You cannot create an object of the static class; therefore the members of the static class can be accessed directly using a class name like **ClassName.MemberName**

- Static classes cannot be **instantiated**.
- All the members of a static class must be static; otherwise the compiler will give an error.
- A static class can contain static variables, static methods, static properties, static operators, static events, and static constructors.
- A static class cannot contain instance members and constructors.
- Indexers and destructors cannot be static
- var cannot be used to define static members. You must specify a type of member explicitly after the static keyword.
- Static classes are sealed class and therefore, cannot be inherited.
- A static class cannot inherit from other classes.
- Static class members can be accessed using `ClassName.MemberName`.
- A static class remains in memory for the lifetime of the application domain in which your program resides.

- The normal class (non-static class) can contain one or more static methods, fields, properties, events and other non-static members.
- It is more practical to define a non-static class with some static members, than to declare an entire class as static.

- Static fields in a non-static class can be defined using the static keyword.
- Static fields of a non-static class is shared across all the instances. So, changes done by one instance would reflect in others.

```
public class Stopwatch
{
    public static int NoOfInstances = 0;

    // instance constructor
    public Stopwatch()
    {
        Stopwatch.NoOfInstances++;
    }
}

class Program
{
    static void Main(string[] args)
    {
        Stopwatch sw1 = new Stopwatch();
        Stopwatch sw2 = new Stopwatch();
        Console.WriteLine(Stopwatch.NoOfInstances); //2

        Stopwatch sw3 = new Stopwatch();
        Stopwatch sw4 = new Stopwatch();
        Console.WriteLine(Stopwatch.NoOfInstances); //4
    }
}
```

- You can define one or more static methods in a non-static class.
- Static methods can be called without creating an object. You cannot call static methods using an object of the non-static class.
- The static methods can only call other static methods and access static members.
- You cannot access non-static members of the class in the static methods.
- **Rules for Static Methods:**
 1. Static methods can be defined using the static keyword before a return type and after an access modifier.
 2. Static methods can be overloaded but cannot be overridden.
 3. Static methods can contain local static variables.
 4. Static methods cannot access or call non-static variables unless they are explicitly passed as parameters.

```
class Program
{
    static int counter = 0;
    string name = "Demo Program";

    static void Main(string[] args)
    {
        counter++; // can access static fields
        Display("Hello World!"); // can call static methods

        name = "New Demo Program"; //Error: cannot access non-static members
        SetRootFolder("C:\\MyProgram"); //Error: cannot call non-static method
    }

    static void Display(string text)
    {
        Console.WriteLine(text);
    }

    public void SetRootFolder(string path) { }
}
```

- A non-static class can contain a parameterless static constructor. It can be defined with the static keyword and without access modifiers like public, private, and protected.
- The static constructor is called only once whenever the static method is used or creating an instance for the first time. Calling the static method second time onwards won't call a static constructor.
- Static members are stored in a special area in the memory called High-Frequency Heap. Static members of non-static classes are shared across all the instances of the class. So, the changes done by one instance will be reflected in all the other instances.

```
public class Stopwatch
{
    // static constructor
    static Stopwatch()
    {
        Console.WriteLine("Static constructor called");
    }
    // instance constructor
    public Stopwatch()
    {
        Console.WriteLine("Instance constructor called");
    }
    // static method
    public static void DisplayInfo()
    {
        Console.WriteLine("DisplayInfo called");
    }
    // instance method
    public void Start() { }
    // instance method
    public void Stop() { }
}
```

```
public class program
{
    public static void Main(string[] args)
    {
        Stopwatch.DisplayInfo(); // static constructor called here
        Stopwatch.DisplayInfo(); // none of the constructors called here
    }
}
```

Microsoft Visual Studio Debug Console

Static constructor called
 DisplayInfo called
 DisplayInfo called

```
public class program
{
    public static void Main(string[] args)
    {
        Stopwatch sw1 = new Stopwatch(); // First static constructor and
                                         // then instance constructor called
        Stopwatch sw2 = new Stopwatch(); // only instance constructor called
        Stopwatch.DisplayInfo();
    }
}
```

Microsoft Visual Studio Debug Console

Static constructor called
 Instance constructor called
 Instance constructor called
 DisplayInfo called

- The static constructor is defined using the static keyword and without using access modifiers public, private, or protected.
- A non-static class can contain one parameterless static constructor. Parameterized static constructors are not allowed.
- Static constructor will be executed only once in the lifetime. So, you cannot determine when it will get called in an application if a class is being used at multiple places.
- A static constructor can only access static members. It cannot contain or access instance members.

- In C#, you can split the implementation of a class, a struct, a method, or an interface in multiple .cs files using the partial keyword.
- The compiler will combine all the implementation from multiple .cs files when the program is compiled.

```
//EmployeeProps.cs
public partial class Employee
{
    public int EmpId { get; set; }
    public string Name { get; set; }
}

//EmployeeMethods.cs
public partial class Employee
{
    //constructor
    public Employee(int id, string name)
    {
        this.EmpId = id;
        this.Name = name;
    }

    public void DisplayEmpInfo()
    {
        Console.WriteLine(this.EmpId + " " + this.Name);
    }
}
```

EmployeeProps.cs contains properties of the Employee class, and EmployeeMethods.cs contains all the methods of the Employee class. These will be compiled as one Employee class.

```
public class Employee
{
    public int EmpId { get; set; }
    public string Name { get; set; }

    public Employee(int id, string name)
    {
        this.EmpId = id;
        this.Name = name;
    }

    public void DisplayEmpInfo()
    {
        Console.WriteLine(this.EmpId + " " + this.Name);
    }
}
```

- All the partial class definitions must be in the same assembly and namespace.
- All the parts must have the same accessibility like public or private, etc.
- If any part is declared abstract, sealed or base type then the whole class is declared of the same type.
- Different parts can have different base types and so the final class will inherit all the base types.
- The Partial modifier can only appear immediately before the keywords class, struct, or interface.
- Nested partial types are allowed.

- Partial classes or structs can contain a method that split into two separate .cs files of the partial class or struct. One of the two .cs files must contain a signature of the method, and other file can contain an optional implementation of the partial method. Both declaration and implementation of a method must have the partial keyword.

```
//EmployeeProps.cs
public partial class Employee
{
    public Employee()
    {
        GenerateEmployeeId();
    }
    public int EmpId { get; set; }
    public string Name { get; set; }

    partial void GenerateEmployeeId();
}

//EmployeeMethods.cs
public partial class Employee
{
    partial void GenerateEmployeeId()
    {
        this.EmpId = random();
    }
}
```

- Partial methods must use the partial keyword and must return void.
- Partial methods can have in or ref but not out parameters.
- Partial methods are implicitly private methods, so cannot be virtual.
- Partial methods can be static methods.
- Partial methods can be generic.

- Sealed Classes
- Sealed methods
- Static Members of a Class
- Static Classes
- Static Methods
- Partial classes
- Partial methods



Thank You